Finite Element, POD-Galerkin, PODNN and PINN Approaches for a Parametric Steady Navier–Stokes Problem

Luciano Scarpino

Abstract

This report summarizes the numerical work carried out on a two-dimensional parametric steady incompressible Navier–Stokes problem. A full order finite element model (FOM) is used as reference solution, and three alternative reduced or learning-based approaches are compared against it: POD-Galerkin, PODNN and PINN. The analysis focuses on the mathematical formulation, the approximation mechanism, the computational cost and the accuracy of each method. POD-Galerkin provides the most accurate reduced approximation in the tested configuration, while PODNN gives the fastest online evaluation by replacing the reduced nonlinear solve with a learned parameter-to-coefficient map. The PINN approach is conceptually different: it avoids the construction of a finite element solver and directly embeds the differential equations into the loss function. In this work, the PINN was mainly used to show the simplicity of a mesh-free physics-informed solver and, at the same time, the practical difficulty of training it to a quantitatively accurate solution.

1 Problem description

The problem considered in this work is the stationary incompressible Navier–Stokes system on the two-dimensional unit square domain

$$\Omega = (0,1)^2.$$

The unknowns are the velocity field

$$u = (u_x, u_y) : \Omega \to \mathbb{R}^2$$

and the pressure field

$$p : \Omega \to \mathbb{R}.$$

The parametric dependence is described by

$$\mu = (\mu_0, \mu_1) \in [1,10] \times [1,3],$$

where μ_0 scales the viscous contribution and μ_1 affects the forcing term. In the numerical tests, the reference visualization is obtained for

$$\mu_0 = 1, \qquad \mu_1 = 2.$$

In strong form, the model can be written as

$$\begin{cases} -\mu_0 \Delta u + (u \cdot \nabla)u + \nabla p = f(x;\mu_1), & \text{in } \Omega, \\ \nabla \cdot u = 0, & \text{in } \Omega, \\ u = 0, & \text{on } \partial\Omega. \end{cases} \tag{1}$$

The first equation is the momentum balance. The term $-\mu_0 \Delta u$ represents viscous diffusion, $(u \cdot \nabla)u$ is the nonlinear convective term, ∇p is the pressure gradient, and $f(x;\mu_1)$ is the external forcing. The second equation imposes incompressibility.

The pressure is defined only up to an additive constant. Therefore, a normalization condition is required. In this implementation, the pressure is fixed at the vertex $(0, 0)$:

$$p(0, 0) = 0.$$

The goal is to solve (1) for different parameter values and then compare a high-fidelity finite element solution (FEM) with reduced-order and neural approximations.

Before using the FOM as reference for all comparisons, the numerical implementation was checked on a configuration with a known reference solution. This preliminary validation step was used to verify that the finite element assembly, the boundary conditions, the nonlinear solver and the pressure normalization behaved as expected. After this validation, the FOM was treated as the reference model for the reduced-order comparisons.

(a) Mesh generated in Python.

(b) Mesh visualization in ParaView.

Figure 1: Finite element mesh used for the full order discretization.

The mesh shown in Figure 1 is an unstructured triangular mesh of the unit square, generated with mesh size parameter

$$h = 0.001.$$

The domain area is

$$|\Omega| = 1.$$

The domain is decomposed into small triangular elements, and the solution is approximated locally on each element by polynomial basis functions. This decomposition is the central idea of FEM: instead of looking for the exact solution in the whole continuous domain, the method builds a finite-dimensional approximation by combining many simple local functions. The quality of the approximation depends on the mesh resolution and on the polynomial degree of the finite element spaces. A finer mesh generally gives a more accurate solution, but it also increases the number of degrees of freedom and therefore the computational cost.

2 Full order finite element model

The full order model is obtained by applying the finite element method to the weak form of (1). The intuition behind FEM is to transform the differential problem into an integral problem and then approximate the unknown fields on the mesh. This is useful because derivatives in the strong form may be difficult to handle directly, while the weak form only requires the solution to satisfy the equations in an averaged sense against suitable test functions. The resulting formulation is naturally adapted to complex geometries, boundary conditions and local mesh refinement.

The natural functional spaces for the continuous problem are

$$V = [H_0^1(\Omega)]^2, \qquad Q = L_0^2(\Omega),$$

where V contains velocity fields satisfying the homogeneous Dirichlet boundary condition and Q contains pressure fields with a suitable normalization.

Multiplying the momentum equation by a test function $v \in V$, multiplying the incompressibility equation by a test function $q \in Q$, and integrating over the domain gives the weak problem: find $(u, p) \in V \times Q$ such that

$$a_\mu(u, v) + c(u, u, v) + b(v, p) = F_\mu(v) \qquad \forall v \in V, \tag{2}$$

$$b(u, q) = 0 \qquad \forall q \in Q. \tag{3}$$

The bilinear and trilinear forms are

$$a_\mu(u, v) = \mu_0 \int_\Omega \nabla u : \nabla v \, dx,$$

$$c(w, u, v) = \int_\Omega (w \cdot \nabla) u \cdot v \, dx,$$

$$b(v, p) = -\int_\Omega p \, \nabla \cdot v \, dx,$$

and the forcing term is

$$F_\mu(v) = \int_\Omega f(x; \mu_1) \cdot v \, dx.$$

The notation $\nabla u : \nabla v$ denotes the Frobenius product between the gradients of the vector fields.

The finite element method replaces the infinite-dimensional spaces V and Q with finite-dimensional spaces

$$V_h \subset V, \qquad Q_h \subset Q.$$

The discrete problem becomes: find $(u_h, p_h) \in V_h \times Q_h$ such that

$$a_\mu(u_h, v_h) + c(u_h, u_h, v_h) + b(v_h, p_h) = F_\mu(v_h) \qquad \forall v_h \in V_h, \tag{4}$$

$$b(u_h, q_h) = 0 \qquad \forall q_h \in Q_h. \tag{5}$$

In practical terms, the velocity and pressure are approximated as linear combinations of finite element basis functions:

$$u_h(x; \mu) = \sum_{i=1}^{N_u} U_i(\mu) \varphi_i(x), \qquad p_h(x; \mu) = \sum_{j=1}^{N_p} P_j(\mu) \psi_j(x).$$

The unknowns of the discrete problem are no longer functions, but the coefficient vectors $\mathbf{u}_h(\mu)$ and $\mathbf{p}_h(\mu)$. The full state is therefore represented as

$$U_h(\mu) = \begin{bmatrix} \mathbf{u}_h(\mu) \\ \mathbf{p}_h(\mu) \end{bmatrix} \in \mathbb{R}^{N_h}.$$

In this work, the FOM dimension is

$$N_h = 6761.$$

Because of the nonlinear convective term, the algebraic system is nonlinear. It can be written in residual form as

$$R_h(U_h; \mu) = 0. \tag{6}$$

Newton's method solves this system iteratively. Given an approximation $U_h^{(k)}$, the update $\delta U_h^{(k)}$ is obtained from

$$J_h(U_h^{(k)}; \mu) \, \delta U_h^{(k)} = -R_h(U_h^{(k)}; \mu), \tag{7}$$

where J_h is the Jacobian of the residual. The next iterate is

$$U_h^{(k+1)} = U_h^{(k)} + \delta U_h^{(k)}.$$

The Newton solver is run with tolerance

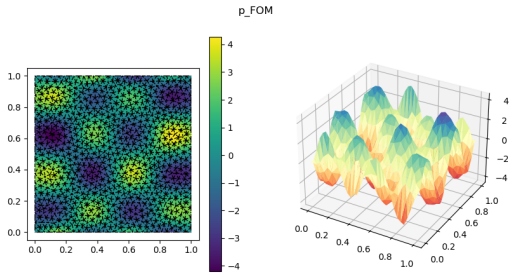
$$10^{-6}$$

and maximum number of iterations equal to

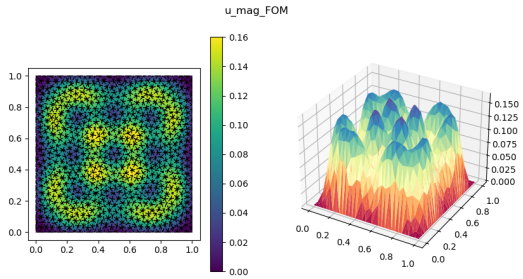
$$10.$$

In the tests reported here, the Newton solver converged in 3 iterations for all sampled parameter values.

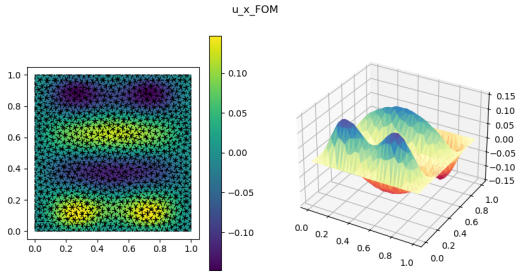
The FOM is the most accurate model considered in this work because it solves the discretized Navier–Stokes equations directly in the full finite element space. Its average computational time over the test set is approximately 0.41–0.42 seconds per parameter value. This cost is acceptable for the present mesh, but it becomes problematic when the mesh is refined, the physical model is more complex or many parameter evaluations are required. This motivates the construction of reduced or surrogate models.



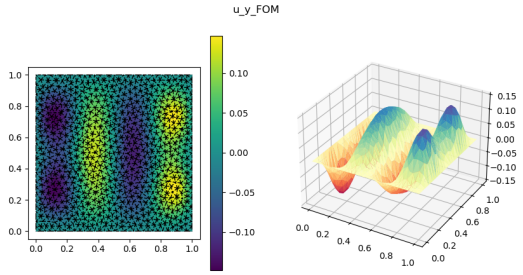
(a) Pressure field.



(b) Velocity magnitude.



(c) Horizontal velocity component u_x.



(d) Vertical velocity component u_y.

Figure 2: FOM solution for a representative parameter configuration: pressure, velocity magnitude and velocity components.

The pressure field in Figure 2 shows a structured oscillatory pattern with positive and negative regions distributed over the domain. The velocity magnitude is zero on the boundary, consistently with the homogeneous Dirichlet condition $u = 0$ on $\partial\Omega$, and reaches its largest values inside the domain. The maximum velocity magnitude is approximately $1.6 \cdot 10^{-1}$, while the pressure ranges approximately between -4 and 4. The horizontal and vertical velocity components, u_x and u_y, show sign-changing structures: positive and negative regions indicate the local direction of the flow along the two coordinate axes. Their combination produces the non-negative velocity magnitude, which highlights where the flow is stronger independently of its direction.

3 POD-Galerkin reduced model

The POD-Galerkin method is an intrusive projection-based reduced order model. Its objective is to approximate the high-dimensional FOM solution in a low-dimensional subspace constructed from representative full order solutions.

In this work, the offline snapshot set is generated by sampling

$$100$$

parameter values uniformly in the range

$$\mu_0 \in [1, 10], \qquad \mu_1 \in [1, 3].$$

The sampling is performed with random seed

$$26.$$

Let

$$U_h(\mu_1), \ldots, U_h(\mu_M)$$

be the FOM snapshots computed for these selected parameter values, with

$$M = 100.$$

These snapshots are collected in the matrix

$$S = \begin{bmatrix} U_h(\mu_1) & U_h(\mu_2) & \cdots & U_h(\mu_M) \end{bmatrix} \in \mathbb{R}^{N_h \times M}.$$

Proper Orthogonal Decomposition searches for an orthonormal basis

$$\Phi_N = \begin{bmatrix} \phi_1 & \phi_2 & \cdots & \phi_N \end{bmatrix} \in \mathbb{R}^{N_h \times N}, \qquad N \ll N_h,$$

that minimizes the average projection error of the snapshots:

$$\min_{\dim(V_N)=N} \sum_{m=1}^{M} \left\| U_h(\mu_m) - P_{V_N} U_h(\mu_m) \right\|^2. \tag{8}$$

The solution is obtained from the dominant left singular vectors of the snapshot matrix, or equivalently from the eigenvalue problem associated with the snapshot correlation matrix. The maximum number of retained POD modes used in the reduction stage is

$$N_{\max} = 100.$$

For the online reduced models compared in this work, the effective reduced dimension is

$$N = 49.$$

The reduced space is

$$V_N = \operatorname{span}\{\phi_1, \ldots, \phi_N\}.$$

The FOM solution is then approximated as

$$U_h(\mu) \approx U_N(\mu) = \Phi_N a(\mu) = \sum_{i=1}^{N} a_i(\mu) \phi_i, \tag{9}$$

where $a(\mu) \in \mathbb{R}^N$ is the vector of reduced coefficients.

The Galerkin projection is obtained by imposing that the full residual is orthogonal to the reduced space:

$$\Phi_N^T R_h(\Phi_N a(\mu); \mu) = 0. \tag{10}$$

This gives a nonlinear reduced system of dimension N instead of N_h. Newton's method is then applied in the reduced space:

$$J_N(a^{(k)}; \mu)\, \delta a^{(k)} = -R_N(a^{(k)}; \mu), \tag{11}$$

$$a^{(k+1)} = a^{(k)} + \delta a^{(k)},$$

where

$$R_N(a;\mu) = \Phi_N^T R_h(\Phi_N a;\mu).$$

This is the key difference between FOM and POD-Galerkin: the nonlinear equations are still solved, but only in a reduced coordinate system.

In this work, the reduced dimension is

$$N = 49,$$

while the full dimension is

$$N_h = 6761.$$

The resulting compression ratio is approximately

$$\frac{N_h}{N} \approx 138.$$

For the Navier–Stokes equations, an important computational issue is the convective term. Since the convection is quadratic in the velocity, it can be precomputed in reduced coordinates. If the reduced velocity is written as

$$u_N = \sum_{i=1}^{N} a_i \phi_i,$$

then the reduced convective contribution has components of the form

$$[C_N(a)]_i = \sum_{j=1}^{N}\sum_{k=1}^{N} C_{ijk} a_j a_k, \tag{12}$$

where

$$C_{ijk} = c(\phi_j, \phi_k, \phi_i).$$

The tensor C_{ijk} is assembled offline. During the online phase, the reduced nonlinear term is evaluated only through sums over the reduced dimension N, without reconstructing or assembling the full finite element operators. This is why POD-Galerkin can be much faster than the FOM while retaining high accuracy.

A further practical issue concerns the reduced forcing term. If the forcing term is available in an affine parametric form,

$$f_h(\mu) \approx \sum_{q=1}^{Q_f} \theta_q^f(\mu) f_q,$$

then the reduced right-hand side can be precomputed offline as

$$f_N(\mu) = \Phi_N^T f_h(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu) \Phi_N^T f_q.$$

However, when the forcing term is not easily available in affine separated form, the online computation of $f_N(\mu)$ may still require operations at the full finite element dimension. A classical strategy to recover an efficient approximation is the Empirical Interpolation Method (EIM), which constructs a separated approximation of the non-affine term by selecting interpolation basis functions and interpolation degrees of freedom.

In this work, the reduced forcing vector is approximated by a small one-hidden-layer neural surrogate of the map

$$\mu_1 \mapsto f_N(\mu_1). \tag{13}$$

This is consistent with the structure of the problem, since the source term depends on μ_1, while μ_0 acts on the viscous operator. During the offline phase, for each sampled value of μ_1, the full FEM right-hand side is assembled and projected onto the reduced basis:

$$f_N(\mu_1) = \Phi_N^T f_h(\mu_1). \tag{14}$$

These projected vectors are then used as training targets for the neural surrogate.

The implemented model is a shallow neural network with a fixed random hidden layer and trainable output weights fitted by ridge regression. More precisely, after normalizing the scalar input μ_1, the hidden features are computed as

$$H(\mu_1) = \tanh\left(\widehat{\mu}_1 W_{\text{in}} + b_{\text{in}}\right),$$

where W_{in} and b_{in} are randomly initialized and then kept fixed. A constant column is added to the hidden feature matrix, producing H_{aug}. The output weights are obtained by solving the ridge regression problem

$$W_{\text{out}} = \arg\min_W \|H_{\text{aug}} W - Y\|_F^2 + \alpha \|W\|_F^2,$$

where the rows of Y are the reduced right-hand sides $f_N(\mu_1)$ assembled and projected offline. This corresponds to an Extreme Learning Machine-type model: the hidden layer is random and fixed, while only the output layer is learned.

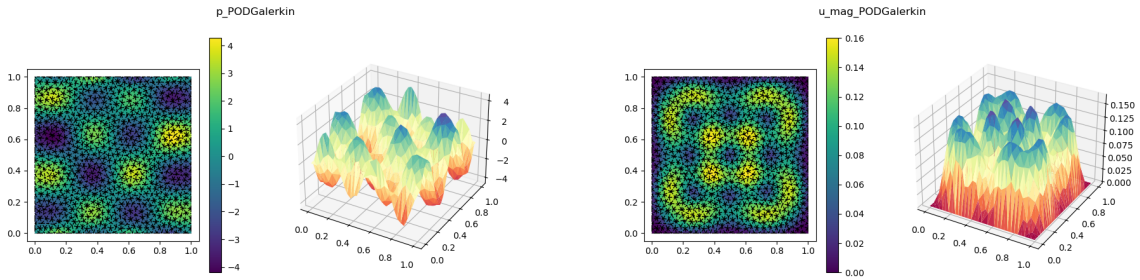
During the online POD-Galerkin phase, the reduced solver evaluates this neural surrogate to obtain $f_N(\mu_1)$, avoiding the assembly and projection of the full FEM right-hand side. This choice was preferred over an EIM strategy mainly for implementation simplicity. EIM is a powerful and more classical reduced-order strategy, but it requires the construction of an empirical interpolation basis, the selection of interpolation degrees of freedom and a more intrusive modification of the treatment of the non-affine source term. Since in this setting the reduced right-hand side depends only on the scalar parameter μ_1, a direct neural approximation of $f_N(\mu_1)$ was simpler to implement and sufficient to preserve the efficiency of the online POD-Galerkin solver in this study.

The POD-Galerkin performance is evaluated on

$$10$$

test parameters, sampled with seed

$$123.$$



(a) Pressure field.

(b) Velocity magnitude.

Figure 3: POD-Galerkin solution for the same representative configuration.

The POD-Galerkin plots are almost indistinguishable from the FOM plots. The pressure range, the internal oscillatory structure and the boundary behavior of the velocity magnitude are preserved. This is expected because the method uses information from FOM snapshots and still solves a physics-based reduced system. Quantitatively, the mean relative error on the full state is

$$6.75 \cdot 10^{-3}.$$

The average online time is

$$6.23 \cdot 10^{-4} \text{ s},$$

which gives an average speedup of about 664 with respect to the FOM. These results also show that the neural approximation of the reduced forcing term is accurate enough in the tested parameter range and does not destroy the quality of the projected reduced model.

4 PODNN reduced model

PODNN combines model reduction and supervised learning. As in POD-Galerkin, the solution is represented in a reduced basis:

$$U_h(\mu) \approx U_N(\mu) = \Phi_N a(\mu). \tag{15}$$

The difference is that PODNN does not compute the reduced coefficients by solving the projected Navier–Stokes equations. Instead, it learns the map

$$\mu = (\mu_0, \mu_1) \mapsto a(\mu) \tag{16}$$

using a neural network. Therefore, the online phase is reduced to two operations: first, the neural network predicts the reduced coefficients; then, the full solution is reconstructed through the POD basis.

The training data are constructed from the same snapshots used for the POD-Galerkin method. For each training parameter μ_m, the FOM solution $U_h(\mu_m)$ is projected onto the POD reduced space. Since the global reduced basis is denoted by Φ_N, the POD coefficient vector is computed by solving the least-squares projection problem

$$a(\mu_m) = \arg\min_{a \in \mathbb{R}^N} \|\Phi_N a - U_h(\mu_m)\|_2^2 . \tag{17}$$

These coefficients are then used as supervised learning targets. The neural network is trained to approximate

$$\mathcal{N}_\theta(\mu_m) \approx a(\mu_m).$$

In this work, the PODNN architecture is a fully connected multilayer perceptron. The input layer has dimension 2, corresponding to the two physical parameters (μ_0, μ_1). The output layer has dimension equal to the number of POD coefficients, namely the dimension of the global reduced basis. In the reported configuration, the reduced basis has dimension

$$N = 49,$$

therefore the neural network predicts 49 reduced coefficients.

The hidden architecture used in the implementation is

$$2 \longrightarrow 128 \longrightarrow 128 \longrightarrow 64 \longrightarrow 49.$$

Each hidden layer is followed by a hyperbolic tangent activation function:

$$\sigma(z) = \tanh(z).$$

Therefore, the network can be written as

$$\mathcal{N}_\theta(\mu) = W_4\sigma\left(W_3\sigma\left(W_2\sigma\left(W_1\widehat{\mu} + b_1\right) + b_2\right) + b_3\right) + b_4, \tag{18}$$

where $\widehat{\mu}$ denotes the normalized input parameter vector, while W_i and b_i are the trainable weights and biases of the network.

Before training, both inputs and outputs are normalized. The parameters are transformed as

$$\widehat{\mu} = \frac{\mu - \mu_{\text{mean}}}{\mu_{\text{std}}},$$

and the POD coefficients are transformed as

$$\widehat{a} = \frac{a - a_{\text{mean}}}{a_{\text{std}}}.$$

This normalization is important because the two parameters and the POD coefficients may have different scales. Training directly on unnormalized quantities may make the optimization harder and may cause the neural network to focus more on variables with larger numerical magnitude.

The training loss is the mean squared error between normalized predicted coefficients and normalized target coefficients:

$$\mathcal{L}_{\text{PODNN}}(\theta) = \frac{1}{M} \sum_{m=1}^{M} \|\mathcal{N}_\theta(\widehat{\mu}_m) - \widehat{a}(\mu_m)\|_2^2 . \tag{19}$$

The network is trained with the Adam optimizer using:

$$\text{epochs} = 5000, \qquad \eta = 10^{-3}, \qquad \text{weight decay} = 10^{-8},$$

with random seed

26.

No mini-batch size is specified in the implementation, therefore the whole training set is used as one batch.

Once the network has been trained, the online prediction is very simple. Given a new parameter value μ, the parameter is normalized, passed through the neural network, and the predicted normalized coefficients are mapped back to the original coefficient scale:

$$a_{\text{PODNN}}(\mu) = a_{\text{std}} \odot \mathcal{N}_\theta(\widehat{\mu}) + a_{\text{mean}}, \tag{20}$$

where $\odot$ denotes component-wise multiplication. The reconstructed solution is then obtained as

$$U_{\text{PODNN}}(\mu) = \Phi_N a_{\text{PODNN}}(\mu). \tag{21}$$

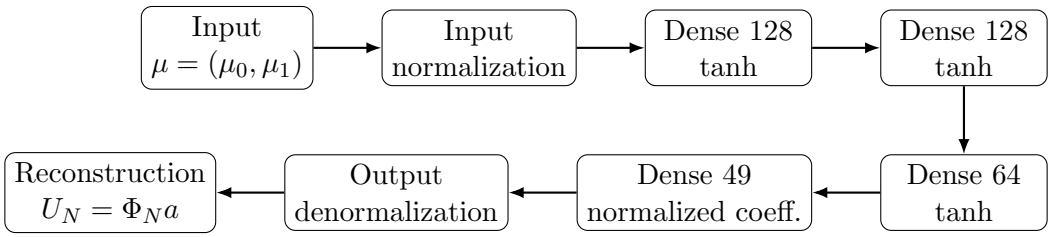


Figure 4: PODNN architecture used in the online phase. The network maps the physical parameters to the POD coefficients, which are then used to reconstruct the full solution through the reduced basis.

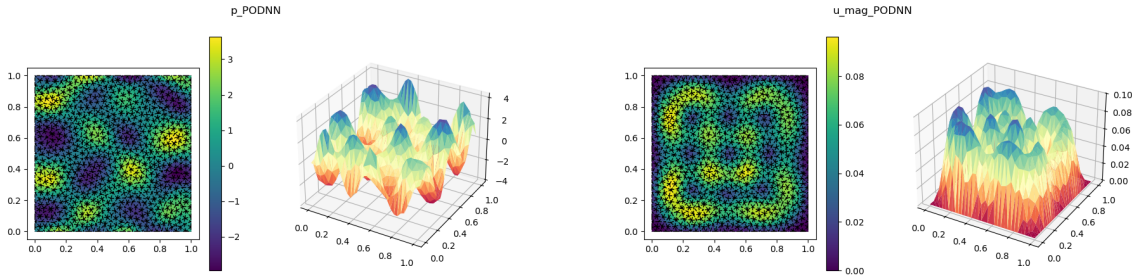
This method is non-intrusive in the online phase. It does not require the assembly of reduced matrices, the construction of reduced convective tensors, the prediction of a reduced forcing term or the solution of a nonlinear reduced system. Therefore, it is particularly useful when the original solver is available only as a black box, or when the goal is to obtain very fast predictions for many parameter values.

However, this advantage comes with a loss of physical structure. PODNN learns the coefficient map only from data. It does not explicitly enforce the Navier–Stokes residual during prediction. Therefore, its accuracy depends strongly on the quality of the training set, the expressiveness of the neural network, the normalization of the data and the generalization capability of the learned map.

The PODNN performance is evaluated on

test parameters, sampled with seed

123.



(a) Pressure field.

(b) Velocity magnitude.

Figure 5: PODNN solution for the same representative configuration.

The PODNN solution preserves the main qualitative structure of the FOM: the pressure still has alternating positive and negative regions, and the velocity magnitude remains zero on the boundary. However, the amplitude of the velocity magnitude is underestimated. The maximum value is close to 0.10, whereas the FOM and POD-Galerkin reach approximately 0.16. This means that the network captures the global spatial pattern but loses part of the intensity of the solution.

Quantitatively, PODNN is less accurate than POD-Galerkin in the present tests. The mean relative full-state error is approximately

$$1.29 \cdot 10^{-1},$$

compared with

$$6.75 \cdot 10^{-3}$$

for POD-Galerkin. However, PODNN is faster in online evaluation. The mean evaluation time is

$$4.57 \cdot 10^{-4} \text{ s},$$

with an average speedup of about 902 with respect to the FOM. Therefore, PODNN is better than POD-Galerkin mainly in terms of online simplicity, non-intrusiveness and raw evaluation speed, but not in accuracy for the current training setup.

5 Physics-Informed Neural Network

The PINN approach is conceptually different from both POD-Galerkin and PODNN. It does not start from a finite element reduced basis and does not require the construction of a snapshot matrix. Instead, a neural network is trained to approximate the solution field directly as a function of space and parameters:

$$\mathcal{N}_\theta(x, y, \mu_0, \mu_1) = \left(u_x^\theta(x, y, \mu), u_y^\theta(x, y, \mu), p^\theta(x, y, \mu)\right). \tag{22}$$

The input of the network is four-dimensional, since it contains the two spatial coordinates and the two physical parameters:

$$(x, y, \mu_0, \mu_1) \in (0, 1)^2 \times [1, 10] \times [1, 3],$$

while the output is three-dimensional:

$$(u_x, u_y, p).$$

In this work, the PINN architecture is an encoder–residual-core–decoder fully connected neural network. Before entering the network, the input variables are normalized to comparable ranges.

The spatial coordinates are mapped from $[0,1]$ to $[-1,1]$, while the parameters are mapped from their physical ranges to $[-1,1]$:

$$\widehat{x} = 2x - 1, \qquad \widehat{y} = 2y - 1,$$

$$\widehat{\mu}_0 = 2\frac{\mu_0 - \mu_{0,\min}}{\mu_{0,\max} - \mu_{0,\min}} - 1, \qquad \widehat{\mu}_1 = 2\frac{\mu_1 - \mu_{1,\min}}{\mu_{1,\max} - \mu_{1,\min}} - 1.$$

This normalization is important because the spatial variables and the physical parameters have different numerical scales. Training directly on the raw variables would make the optimization less well conditioned.

The PINN model is built with:

$$\text{width} = 128, \qquad \text{latent dimension} = 64, \qquad \text{number of residual blocks} = 4.$$

The first part of the network is an encoder:

$$4 \longrightarrow 128 \longrightarrow 64,$$

with SiLU activation functions. The encoder maps the normalized input into a latent representation of dimension 64. Then, the latent vector is lifted back to a feature space of dimension 128:

$$64 \longrightarrow 128.$$

The central part of the architecture is a residual fully connected core composed of four residual blocks. Each block keeps the same feature dimension 128 and has the form

$$z_{\text{out}} = \sigma\left(z + B(z)\right), \tag{23}$$

where

$$B(z) = W_2\sigma(W_1 z + b_1) + b_2,$$

and σ is the SiLU activation function. The residual connection helps the training because each block learns a correction to the incoming features instead of learning the whole transformation from scratch. This is useful in a PINN setting, where the optimization problem is usually harder than in standard supervised learning.

Finally, the decoder maps the residual features to the physical variables:

$$128 \longrightarrow 128 \longrightarrow 3.$$

The three raw outputs are

$$(\widetilde{u}_x, \widetilde{u}_y, \widetilde{p}).$$

The pressure output is used directly:

$$p^\theta = \widetilde{p}.$$

For the velocity field, a hard boundary enforcement is introduced. The raw velocity components are multiplied by the factor

$$g(x,y) = x(1-x)y(1-y).$$

Therefore,

$$u_x^\theta(x,y,\mu) = g(x,y)\widetilde{u}_x(x,y,\mu), \qquad u_y^\theta(x,y,\mu) = g(x,y)\widetilde{u}_y(x,y,\mu). \tag{24}$$

Since $g(x,y) = 0$ on the boundary of the unit square, this construction imposes

$$u^\theta = 0 \qquad \text{on } \partial\Omega$$

by construction. This is a useful architectural choice because the no-slip velocity condition is not only encouraged by the loss, but embedded directly into the network output.

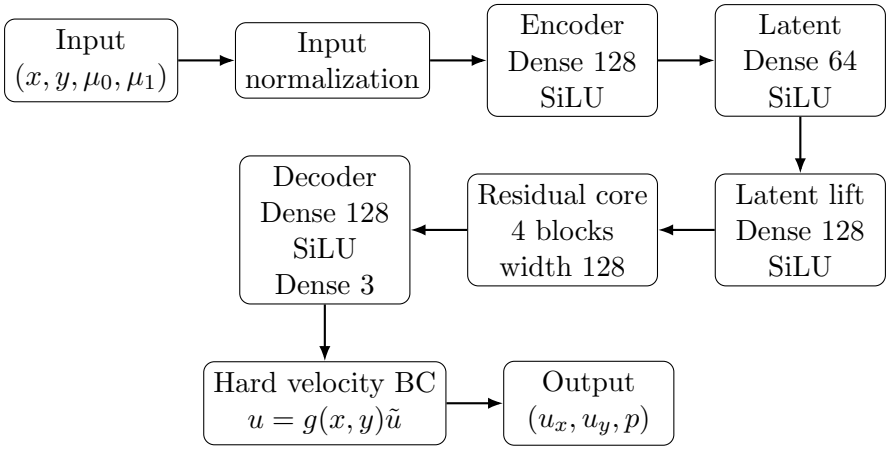


Figure 6: PINN architecture used in this work. The network maps spatial coordinates and physical parameters to the velocity-pressure solution. The velocity boundary condition is imposed through the multiplicative factor $g(x, y) = x(1 - x)y(1 - y)$.

The differential equation is imposed by automatic differentiation. The derivatives appearing in the Navier–Stokes equations are computed directly from the network output with respect to the spatial variables. The momentum residual is

$$r_m^\theta = -\mu_0 \Delta u^\theta + (u^\theta \cdot \nabla) u^\theta + \nabla p^\theta - f(x; \mu_1), \tag{25}$$

and the incompressibility residual is

$$r_d^\theta = \nabla \cdot u^\theta. \tag{26}$$

More explicitly, the two momentum residual components are

$$r_x^\theta = -\mu_0 \Delta u_x^\theta + u_x^\theta \frac{\partial u_x^\theta}{\partial x} + u_y^\theta \frac{\partial u_x^\theta}{\partial y} + \frac{\partial p^\theta}{\partial x} - f_x,$$

$$r_y^\theta = -\mu_0 \Delta u_y^\theta + u_x^\theta \frac{\partial u_y^\theta}{\partial x} + u_y^\theta \frac{\partial u_y^\theta}{\partial y} + \frac{\partial p^\theta}{\partial y} - f_y,$$

while the divergence residual is

$$r_{\mathrm{div}}^\theta = \frac{\partial u_x^\theta}{\partial x} + \frac{\partial u_y^\theta}{\partial y}.$$

All these derivatives are computed through PyTorch automatic differentiation. This is one of the main practical advantages of PINNs: the residual of the strong form can be evaluated without assembling finite element matrices.

The PINN loss used in the implementation is built as a weighted combination of four terms:

$$\mathcal{L}(\theta) = \lambda_{\mathrm{PDE}} \mathcal{L}_{\mathrm{PDE}} + \lambda_{\mathrm{div}} \mathcal{L}_{\mathrm{div}} + \lambda_{\mathrm{b}} \mathcal{L}_{\mathrm{boundary}} + \lambda_{\mathrm{p}} \mathcal{L}_{\mathrm{pressure\ anchor}}. \tag{27}$$

The PDE loss is computed at interior collocation points:

$$\mathcal{L}_{\mathrm{PDE}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \left(|r_x^\theta(x_i, \mu_i)|^2 + |r_y^\theta(x_i, \mu_i)|^2 \right),$$

and the divergence loss is

$$\mathcal{L}_{\mathrm{div}} = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_{\mathrm{div}}^\theta(x_i, \mu_i)|^2.$$

The boundary loss is

$$\mathcal{L}_{\mathrm{boundary}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left\| u^\theta(x_i^b, \mu_i) \right\|_2^2,$$

where x_i^b are boundary collocation points. In the present implementation, the velocity boundary condition is already imposed by the hard factor $g(x, y)$, but the boundary term can still be included in the loss as an additional consistency term.

The pressure anchor is necessary because the pressure is defined up to an additive constant. It is imposed by penalizing the pressure value at $(0, 0)$:

$$\mathcal{L}_{\text{pressure anchor}} = \frac{1}{N_p} \sum_{i=1}^{N_p} |p^\theta(0, 0, \mu_i)|^2.$$

The model is trained with Adam using

$$\text{epochs} = 10000, \qquad N_{\text{interior}} = 4096, \qquad N_{\text{boundary}} = 1024,$$

$$\eta = 10^{-3}, \qquad \text{weight decay} = 10^{-8}.$$

At each training stage, interior and boundary collocation points are sampled to evaluate the physics-informed loss terms. The loss weights are either loaded from a previously tuned configuration or, if no tuned file is found, initialized with the default values

$$\lambda_{\text{PDE}} = 1, \qquad \lambda_{\text{div}} = 1, \qquad \lambda_{\text{boundary}} = 10, \qquad \lambda_{\text{pressure anchor}} = 1.$$

For the reported comparison, the tuned configuration used after the lambda-search step is

$$\lambda_{\text{PDE}} = 1, \qquad \lambda_{\text{div}} = 10, \qquad \lambda_{\text{boundary}} = 1, \qquad \lambda_{\text{pressure anchor}} = 0.1.$$

The PINN is evaluated on the same number of test parameters used for the other methods:

$$10,$$

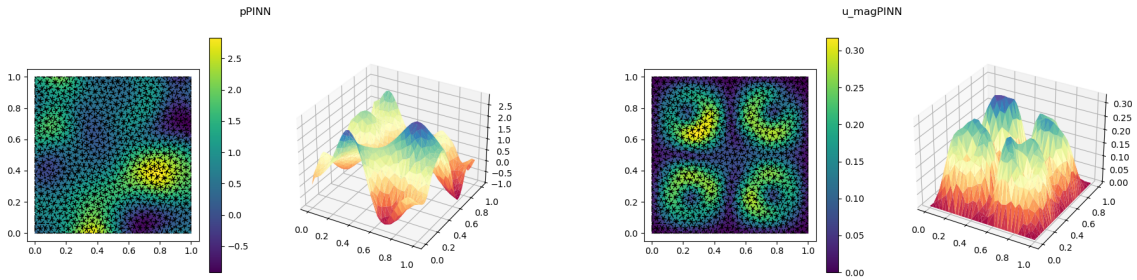
sampled with seed

$$123.$$

The main purpose of the PINN implementation in this work was not to build a fully optimized PINN solver. Rather, the objective was to show the contrast between the apparent simplicity of the method and the real difficulty of obtaining accurate results. From an implementation point of view, the solver is simple because no finite element weak formulation, no assembly of matrices and no Newton solver are required. Once the strong form of the PDE is written, the network can be trained by minimizing residuals evaluated through automatic differentiation.

At the same time, the training problem is much more delicate than the implementation may suggest. The different components of the loss have different physical meanings and different numerical scales. The PDE residual, the divergence-free constraint, the boundary condition and the pressure normalization must all be balanced. If the weights λ are not chosen properly, the network may reduce one part of the residual while producing an unsatisfactory solution elsewhere. For example, a large PDE weight may improve the interior momentum residual while failing to control the divergence or the pressure level; conversely, a large boundary weight may improve the no-slip condition without producing a globally accurate solution.

Therefore, the PINN part of this work should be interpreted as an exploratory test. It shows that the solver can be written in a compact and mesh-free way, but also that achieving a quantitatively satisfactory Navier–Stokes solution requires a much deeper training strategy. Such a systematic training study is possible, but it does not belong to the main nature of the present work, which is centered on comparing FOM, POD-Galerkin and PODNN reduced-order strategies.



(a) Pressure field.

(b) Velocity magnitude.

Figure 7: PINN solution for the same representative configuration.

The PINN plots show that the model has learned some qualitative features of the solution, such as the vanishing velocity magnitude near the boundary and the presence of internal spatial structures. Nevertheless, the field amplitudes and spatial patterns differ significantly from the FOM. The pressure range is smaller and shifted with respect to the FOM, and the velocity magnitude reaches values around 0.30, which is almost twice the FOM scale. This indicates that the present PINN is not yet quantitatively reliable as a surrogate for the FOM.

A tuning experiment was performed on the loss weights. The best configuration among those tested was

$$\lambda_{\text{PDE}} = 1, \qquad \lambda_{\text{div}} = 10, \qquad \lambda_{\text{boundary}} = 1, \qquad \lambda_{\text{pressure anchor}} = 0.1,$$

with score 13.1371. This suggests that the divergence constraint plays an important role in the training. However, the global comparison with the FOM remains poor: the mean relative full-state error is about 1.0868, and the average evaluation time is about 24.35 seconds, much slower than the FOM in this implementation. These results do not imply that PINNs are unsuitable in general; they show that the current training setup is not sufficient for this specific quantitative comparison.

6 Comparison with the FOM

Table 1 summarizes the main quantitative results. All errors are relative errors with respect to the FOM solution. The FOM itself is not reported as an approximation method because it is the reference model.

Table 1: Mean performance over the test set.

Metric	POD-Galerkin	PODNN	PINN
Relative error on full state $U = (u, p)$	0.006753	0.1292	1.087
Relative error on u_x	0.002012	0.1856	0.6533
Relative error on u_y	0.002675	0.2102	1.457
Relative error on $\lvert u \rvert$	0.001617	0.1475	0.7279
Relative error on p	0.006754	0.1292	1.087
Online/evaluation time [s]	0.0006234	0.0004572	24.35
Speedup w.r.t. FOM	664.3	901.9	0.01731

The best overall approximation is obtained by POD-Galerkin. It gives errors below 1% on average for the full state and pressure, while also achieving a speedup larger than 600. This is the expected behavior when the solution manifold is well represented by the POD basis and the reduced operators are correctly precomputed. POD-Galerkin remains physically structured because it solves a projected version of the original finite element equations. Its main limitation is that it is intrusive: it requires access to the finite element operators and to the assembly structure

of the original solver. In this implementation, the reduced forcing term is not assembled online through the full model, but approximated by a shallow neural surrogate trained offline on projected FEM right-hand sides. This makes the method slightly less purely intrusive than a standard affine POD-Galerkin implementation, while preserving most of its projection-based structure.

PODNN has a different profile. It is less accurate than POD-Galerkin, but it is the fastest method in the online phase. Its average speedup is about 902, and it avoids Newton iterations completely. This makes it useful when fast repeated evaluations are more important than very high accuracy, or when the original model is available only through generated data. The present PODNN still underestimates some velocity amplitudes, suggesting that more training data, a better network architecture, improved normalization or a different loss definition could improve the accuracy.

The PINN is the least accurate and slowest approach in the reported experiments. This result must be interpreted carefully. The value of the PINN experiment is not that it provides the best numerical approximation, but that it shows a radically different way of constructing a solver. The strong form of the PDE can be embedded directly into the loss, avoiding the finite element machinery. However, this simplicity in formulation is compensated by a difficult training problem. The current PINN did not reach satisfactory accuracy because the optimization, the collocation sampling and the loss balancing were not developed deeply enough. A more mature PINN implementation would require adaptive loss weights, more systematic tuning of the λ coefficients, a better sampling strategy and possibly a longer or staged training procedure.

7 Conclusions

This work compares four levels of modeling for a parametric Navier–Stokes problem. The FOM provides the reference finite element solution and was preliminarily validated on a known solution before being used for comparison. It is accurate and physically faithful, but its cost scales with the full finite element dimension.

POD-Galerkin gives the best compromise between accuracy and efficiency in this setting. It uses snapshots to construct a reduced space and then projects the governing equations onto that space. The method is highly accurate because it preserves the structure of the finite element residual, and it is fast because all computations are performed in a low-dimensional space. In this work, the reduced right-hand side was approximated by a shallow neural surrogate of the map $\mu_1 \mapsto f_N(\mu_1)$ instead of using an EIM strategy. This choice simplified the implementation and avoided the construction of an empirical interpolation procedure, while still allowing an efficient online evaluation. POD-Galerkin is therefore the natural choice when the original finite element solver and its operators are accessible, and when a structured reduced model is desired.

PODNN is preferable when the goal is a very fast non-intrusive surrogate. It does not solve a reduced nonlinear system, but directly predicts the reduced coefficients from the parameters. In the current results, it is less accurate than POD-Galerkin, but faster and easier to deploy once trained. This makes it particularly interesting for black-box settings, data-driven surrogate modeling and applications requiring many rapid evaluations.

PINNs represent a conceptually different strategy. They do not rely on snapshots and reduced bases in the same way as POD methods, but try to learn the solution while enforcing the governing equations through the loss. In this work, the PINN was mainly used to highlight the simplicity of a solver that avoids finite element assembly and uses automatic differentiation to impose the PDE. At the same time, the obtained results show that this simplicity comes with a significant training complexity. The loss terms must be balanced carefully, the boundary and domain residuals must be controlled simultaneously, and the optimization procedure must be much more systematic to obtain a satisfactory solution. Therefore, PINNs have good potential, but their full development is beyond the main scope of this work.

Overall, POD-Galerkin is the most reliable method for the present problem, PODNN is the most efficient online surrogate, and PINN remains a promising but more demanding direction that requires further development.